

Milestone 1: Technical Understanding and Research Framing

Handling Execution Overruns in Hard Real-Time Control Systems

Christian Percival
Electronic Engineering
Hochschule Hamm-Lippstadt
Lippstadt, Germany

I. PURPOSE OF THIS MILESTONE

This milestone documents the initial technical understanding and research framing for the seminar topic *Handling Execution Overruns in Hard Real-Time Control Systems*. The goal is not to produce a polished final paper, but to show that the assigned work has been technically understood. Therefore, this document uses structured notes, equations, tables, concise explanations, and open questions.

II. REFERENCES USED IN THIS MILESTONE

- Main technical paper: M. Caccamo, G. Buttazzo, and L. Sha, “Handling Execution Overruns in Hard Real-Time Control Systems,” *IEEE Transactions on Computers*, 2002.
- Official background reference: G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed., Springer, 2024.
- The main paper provides the specific overrun-handling method, while the book provides the general theoretical background on hard real-time systems, EDF scheduling, CBS, resource sharing, and overload/overrun management.

III. PROBLEM STATEMENT AND MOTIVATION

Hard real-time systems must produce correct results before strict deadlines. In these systems, correctness depends not only on the computed value, but also on the time at which the value is produced. A late result can be useless or dangerous, especially in control applications such as robotics, autonomous systems, industrial automation, automotive systems, and aerospace systems.

The seminar topic focuses on the problem of *execution overruns*. An execution overrun occurs when a task executes longer than its expected or reserved computation time. This is different from simply finishing earlier than the worst-case execution time (WCET). Finishing earlier than WCET creates unused processor time, whereas an overrun means that the task exceeds its planned execution budget.

A. Main Problem

The core technical problem can be stated as follows:

How can a hard real-time control system improve processor utilization and control performance during normal execution while still guaranteeing schedulability when occasional execution overruns occur?

B. Why WCET-Only Scheduling is Inefficient

Traditional hard real-time schedulability analysis often uses the WCET of every task. This is safe, but conservative.

- If every task is scheduled according to its WCET, timing guarantees are easier to prove.
- However, many tasks normally execute much faster than their WCET.
- Reserving WCET in every period can waste processor bandwidth.
- In control systems, wasted bandwidth can reduce the achievable sampling frequency.
- Lower sampling frequency can reduce control performance.

The key motivation is therefore the tension between:

- **safety and predictability:** guarantee deadlines even in the worst case,
- **efficiency and performance:** avoid wasting processor time when tasks execute normally.

C. Motivating Example: Visual Tracking

The paper motivates the problem using visual tracking in control systems. A visual tracking task may normally find an object quickly inside a small predicted search window. However, when the object moves outside the predicted region, the search area must be expanded. This increases the execution time.

Example behavior:

- Normal case: object found in small search window.
- Medium case: object found after expanding the search region.

- Worst case: object not found until a full image scan is performed.

This creates a task with a short normal execution time but a much larger WCET. Such behavior is common in robotics, visual servoing, radar tracking, and sensor-based control systems.

IV. CORE TECHNICAL IDEA

The main idea of the assigned paper is to combine control-performance optimization with real-time scheduling guarantees.

Instead of always scheduling tasks using their WCET, the approach uses:

- normal computation time for improving normal-case task frequencies,
- WCET information for guaranteeing hard deadlines,
- server-based scheduling for isolating overruns,
- rate adaptation for improving control performance.

The essential idea is:

Use the processor more efficiently during normal execution, but ensure that occasional overruns remain bounded and do not cause other tasks to miss deadlines.

V. IMPORTANT TERMINOLOGY

TABLE I
IMPORTANT TERMS FOR UNDERSTANDING EXECUTION OVERRUN HANDLING

Term	Meaning
Hard real-time task	A task for which missing a deadline is unacceptable.
WCET	Worst-case execution time; the maximum assumed execution time used for safe analysis.
Normal computation time	The typical computation time required during normal operation.
Execution overrun	A situation where a task executes longer than its expected or reserved execution budget.
EDF	Earliest Deadline First; a dynamic-priority scheduler that executes the job with the earliest deadline first.
CBS	Constant Bandwidth Server; a scheduling server that assigns a task a fixed processor bandwidth.
CBS _{hd}	A hard-deadline extension of CBS used for more efficient overrun handling.
PLI	Performance Loss Index; a metric that models the loss in control performance as a function of sampling frequency.
Temporal isolation	A property ensuring that one overrunning task cannot steal unlimited processor time from other tasks.

VI. FORMAL MODEL AND MATHEMATICAL BASICS

Each periodic control task is modeled as:

$$\tau_i(WCET_i, c_i^n, f_i^{min}, \Delta J_i(f_i))$$

where:

- $WCET_i$ is the worst-case execution time of task τ_i ,

- c_i^n is the normal computation time of task τ_i ,
- f_i^{min} is the minimum safe execution frequency,
- $\Delta J_i(f_i)$ is the Performance Loss Index of the task.

A. Task Utilization

For a periodic task, utilization can be expressed as:

$$U_i = C_i f_i$$

where C_i is the computation time and f_i is the execution frequency.

The total utilization of a task set is:

$$U = \sum_{i=1}^n U_i$$

For EDF scheduling on a uniprocessor, a common schedulability condition is:

$$\sum_{i=1}^n U_i \leq 1$$

This means that the total processor demand must not exceed the available processor capacity.

B. Performance Loss Index

The Performance Loss Index models how control quality changes with task frequency. The paper uses the following exponential approximation:

$$\Delta J_i(f_i) = \alpha_i e^{-\beta_i f_i}$$

where:

- f_i is the sampling or execution frequency of the control task,
- α_i is the magnitude coefficient,
- β_i is the decay rate,
- $\Delta J_i(f_i)$ is the performance loss.

Interpretation:

- Larger f_i usually means better control performance.
- Larger f_i also means higher CPU utilization.
- The scheduling problem is therefore an optimization problem under timing constraints.

The total system performance loss is:

$$\Delta J(f_1, \dots, f_n) = \sum_{i=1}^n w_i \Delta J_i(f_i)$$

where w_i represents the relative importance of each control task.

VII. HANDLING EXECUTION OVERRUNS

A. Basic Overrun Situation

A task is expected to execute for its normal computation time:

$$c_i^n$$

but in some cases it may require:

$$c_i > c_i^n$$

If the task exceeds its reserved budget, an overrun occurs. The overrun amount is:

$$c_i - c_i^n$$

If the task is allowed to continue without control, it may delay other tasks and cause deadline misses. Therefore, the scheduler must manage the extra execution time.

B. Local Overrun Handling

The paper proposes that overruns should be handled locally where possible. This means that if task τ_i overruns, the effect should mainly be limited to τ_i itself.

Local overrun handling aims to prevent the following problem:

- Task τ_i executes longer than expected.
- It consumes more CPU time than planned.
- Other tasks are delayed.
- Deadlines of unrelated tasks may be missed.

The desired behavior is:

- the overrunning task is delayed or has its deadline postponed,
- other tasks keep their expected timing behavior,
- hard schedulability guarantees are preserved.

C. Bandwidth Reservation

Each task is assigned a reserved bandwidth:

$$U_i = f_i^{opt} c_i^n$$

where:

- f_i^{opt} is the optimized frequency,
- c_i^n is the normal computation time.

This means that the task receives a controlled fraction of the processor. If it requests more computation than expected, its execution is regulated by the server mechanism.

D. Constant Bandwidth Server

A CBS is described by:

$$(Q_s, T_s)$$

where:

- Q_s is the server budget,
- T_s is the server period.

The server bandwidth is:

$$U_s = \frac{Q_s}{T_s}$$

The basic purpose of CBS is temporal isolation. It prevents a task from consuming more than its reserved bandwidth over time.

Simplified behavior:

```
if task uses available budget:
    task executes normally
else if budget is exhausted:
    postpone server deadline
    recharge budget according to server rule
```

E. CBShd

The paper introduces CBShd as an extension for hard-deadline tasks. The reason is that the standard CBS can postpone deadlines too far when the remaining overrun is small.

CBShd improves this by considering the remaining computation time.

Simplified rule:

```
if remaining_time >= server_budget:
    recharge full budget
    postpone deadline by server period
else:
    recharge only remaining_time
    postpone deadline only as much as necessary
```

Technical interpretation:

- Standard CBS protects the system, but may be pessimistic.
- CBShd handles partial overruns more accurately.
- Smaller unnecessary deadline postponements can improve responsiveness.

VIII. ASSUMPTIONS AND SCOPE LIMITATIONS

The approach depends on several assumptions:

- Tasks are periodic control tasks.
- Each task has a known or bounded WCET.
- Each task has an estimated normal computation time.
- Minimum safe task frequencies are known.
- The task set is scheduled under EDF or a compatible server-based method.
- Overruns are bounded by WCET.
- Remaining computation time can be estimated or bounded.
- The control-performance loss can be represented by a PLI model.

Possible limitations:

- WCET estimation can be difficult on modern processors.
- Normal execution time may depend strongly on input data.
- Server-based scheduling introduces runtime overhead.
- Smaller budgets improve precision but increase the number of deadline updates.

- Implementation is easier in a real-time kernel than in a general-purpose operating system.
- The mathematical model may not fully capture complex modern image-processing or machine-learning workloads.

IX. RUNTIME, RESOURCE, AND SCALABILITY ASPECTS

A. Runtime Overhead

The algorithm introduces overhead because the scheduler must manage budgets, deadlines, and possible budget replenishments.

Important tradeoff:

- Small server budget:
 - better overrun precision,
 - better adaptation,
 - more deadline updates,
 - higher scheduler overhead.
- Large server budget:
 - lower scheduler overhead,
 - less precise overrun handling,
 - potentially more pessimistic response.

B. Resource Sharing

In real embedded systems, tasks often share resources such as memory buffers, sensors, communication buses, or hardware peripherals. Shared resources can introduce blocking times.

The official background book discusses resource access protocols such as:

- Priority Inheritance Protocol,
- Priority Ceiling Protocol,
- Stack Resource Policy.

For this milestone, the main point is:

Overrun handling becomes more complex when tasks share resources, because an overrunning task may hold a resource while its server budget is exhausted.

C. Scalability

The approach scales conceptually because each task can be assigned its own bandwidth reservation. However, practical scalability depends on:

- the number of tasks,
- the number of shared resources,
- the frequency of overruns,
- the accuracy of WCET estimates,
- the overhead of the scheduler implementation.

X. RELEVANCE FOR EMBEDDED SYSTEMS

This topic is relevant for embedded systems because many embedded applications combine strict timing requirements with variable computation times.

Relevant application areas:

- robotic vision,
- autonomous vehicles,

- radar tracking,
- industrial control,
- cyber-physical systems,
- sensor fusion systems.

In these systems, always scheduling for WCET may waste resources, but ignoring overruns may cause missed deadlines. Overrun handling is therefore a key mechanism for balancing efficiency and predictability.

XI. PLANNED OWN IMPLEMENTATION

To make the seminar more practical, a simplified implementation is planned. The purpose is not to build a full real-time kernel scheduler, but to demonstrate the overrun problem and a simplified overrun-handling strategy.

A. Experiment Idea

The planned experiment is an image-processing task inspired by the visual-tracking motivation of the paper.

Basic idea:

- Generate synthetic images containing a colored object.
- Search first inside a small predicted region.
- If the object is not found, expand the search area.
- If still not found, search the full image.
- Measure execution time for every frame.
- Detect when execution time exceeds the normal budget.
- Compare different scheduling strategies.

B. Image Difficulty Levels

TABLE II
PLANNED IMAGE-PROCESSING DIFFICULTY LEVELS

Case	Expected behavior
Easy	Object is inside the predicted small window; short execution time.
Medium	Object is outside the small window but inside a larger region.
Hard	Object is far from the prediction; full image search required.
Worst case	Object is missing; the full search must be completed.

C. Strategies to Compare

TABLE III
PLANNED SCHEDULING STRATEGIES

Strategy	Expected observation
WCET-based scheduling	Safe but inefficient; lower task frequency and unused time in normal cases.
Normal-budget scheduling	Efficient in normal cases but deadline misses may occur during overruns.
Simplified overrun handling	Detects execution overruns and accounts for extra execution time using a budget/deadline rule.

D. Measurements

The experiment should log:

- image/frame number,
- difficulty level,
- search mode,
- execution time,
- normal execution budget,
- estimated WCET,
- overrun flag,
- deadline miss flag,
- scheduling strategy.

E. Simplified Overrun Rule

A simplified rule inspired by CBS/CBS_{hd} can be implemented as:

```
normal_budget = expected normal execution time
bandwidth = normal_budget / task_period
```

```
if execution_time <= normal_budget:
    overrun = False
else:
    overrun = True
    extra_time = execution_time - normal_budget
    deadline_extension = extra_time / bandwidth
    adjusted_deadline = original_deadline + deadline_extension
```

This experiment will be described clearly as a simulation or demonstration, not as a full real-time operating system implementation.

XII. OPEN QUESTIONS FOR FEEDBACK

- How much detail should be included about CBS and CBS_{hd} in the final five-page paper?
- Should the final paper focus more on the theoretical scheduler or on the image-processing overrun example?
- Is a Python/OpenCV simulation acceptable as an implementation demonstration?
- How should WCET be estimated for the experiment: maximum observed execution time or a deliberately constructed worst case?
- Should resource sharing and SRP be explained in detail, or only briefly mentioned?
- Should the implementation use synthetic images only, or also downloaded real images?
- How should the presentation visualize deadline postponement and budget exhaustion?

XIII. MILESTONE 1 CHECKLIST REFLECTION

REFERENCES

- [1] M. Caccamo, G. Buttazzo, and L. Sha, "Handling Execution Overruns in Hard Real-Time Control Systems," *IEEE Transactions on Computers*, vol. 51, no. 7, pp. 835–849, 2002.
- [2] G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed. Springer, 2024.
- [3] C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

TABLE IV
MILESTONE 1 SELF-CHECK

Checkpoint	Status
Technical problem stated without copying the abstract	Yes
Main contribution explained in own words	Yes
Central algorithm/model identified	Yes
Assumptions and scope limitations identified	Yes
Runtime/resource/scalability aspects considered	Yes
Open questions documented	Yes
Relevance for embedded systems explained	Yes
Own implementation direction identified	Yes

- [4] L. Abeni and G. Buttazzo, "Integrating Multimedia Applications in Hard Real-Time Systems," in *Proceedings of the IEEE Real-Time Systems Symposium*, 1998.
- [5] T. P. Baker, "Stack-Based Scheduling of Real-Time Processes," *Real-Time Systems*, vol. 3, no. 1, pp. 67–99, 1991.